

Desenvolva um projeto levando em conta as práticas que fizemos durante o curso. O projeto será feito usando FreeRTOS – podem usar rotinas desenvolvidas em aulas. Conecte 2 kits via UART.

São fornecidos os seguintes dados / requisitos:

* Para *piscar* os LEDs ou *pulsar* intermitentemente o BUZZER use $dt = 200$ ms ON/OFF.

* Comunicação via **UART**: **RX** = **PA10**, **TX** = **PA9** (BAUD RATE = 115.200 / 8 bits / no parity/ 1 stop Bit). Use DMA.

Detalhes do projeto:

a) A partir do RESET do MCU: imediatamente após você acionar o RESET, o microcontrolador vai:

a.1) disparar um **cronômetro crescente** que conta de zero até 9:59:9 e retorna ao zero, continuamente.

a.2) disparar um **conversor ADC** que amostra sinais também continuamente, vamos ler um valor analógico no pino **PA0**. A taxa de amostragem do ADC será de 2 samples/s.

a.3) execute a rotina de teste para ver se os LEDs do painel de display (4x7-SEG) estão funcionando no *shield*. O teste deve ligar todos os segmentos no display: **8.8.8.8**.

a.4) Depois de 3s com tudo aceso, seu kit constantemente fará PING (envie a string “oper?” 4 vezes por segundo). O outro kit deve responder “oper!”, isso indica comunicação viável. Caso não obtenha resposta, entre numa rotina de “erro de conexão”. (Claro, o outro kit fará PING e seu kit deve responder). Se seu kit recebe resposta do outro, mude para outra etapa.

a.5) O PING ocorre durante toda a operação, e se ocorrer falta de resposta, o programa entra na rotina “erro de conexão”; mostre no seu display a mensagem **Con**; e o programa espera outro reset.

b) Etapa 2 – interação com o usuário:

b.1) Ao acionarmos o botão **A1** (**PA1**), o display de 7-SEG alterna o valor do cronômetro e do ADC a cada 4 segundos (**2v/4s: 2 valores/4 s** – ou seja: cronômetro por 4s, ADC por 4s, loop contínuo).

b.1.1) O cronômetro será mostrado no painel display na forma: **minutos** (1 dígito), “**pto** decimal” ligado, **segundos** (com 2 dígitos), “**pto** decimal” ligado, **décimos** de segundo (1 dígito); ex: **3.58.4**;

b.1.2) O ADC será mostrado na forma: **unidade**, “**pto**” ligado, **centésimo**, **décimo**, e **milésimo** de Volt com pontos desligados (ou seja, o milésimo é o menos significativo, resolução = 0.001 Volt); ex: **1.027**;

b.2) para distinguir o valor mostrado no display, pisque o **LED 1** (**PB15**) quando mostrar o cronômetro, e pisque o **LED 2** (**PB14**) quando mostrar o valor do ADC. O formato dos valores no display 7-SEG já indica qual medida está no visor, mas usaremos os LEDs piscando também;

c) Ao acionarmos o botão **A2** (**PA2**), *seu kit inicia comunicação com outra placa* e deve, a partir daí, enviar dados para outro kit. Você passa a ‘*requisitar serviço*’ da outra placa. Portanto, você vai **enviar uma mensagem requisitando serviços** do outro kit.

c.1) Se você acionar **A2** antes de acionar **A1** seus valores serão mostrados no outro kit, mas não no seu kit; ou seja, sua placa continuará a mostrar **8.8.8.8**, embora seus dados sejam mostrados remotamente.

d) quando o kit do colega solicitar serviço (para mostrar os dados dele), **sua placa NÃO pode recusar** a solicitação e deve mudar o modo de mostrar dados para **4 valores/2 s** (**4v/2s**); mostrando, repetidamente:

* por 2 segundos, piscando o **D1**, deve mostrar seu próprio cronômetro (ou **8.8.8.8**);

** por 2 segundos, piscando o **D2**, deve mostrar seu próprio ADC (ou **8.8.8.8**);

*** por 2 segundos, piscando o **D3** - **PB13**, deve mostrar o valor do cronômetro do colega;

**** por 2 segundos, piscando o **D4** - **PB12**, deve mostrar o valor do ADC do colega;

d.1) Se você ainda não acionou **A1** e sua placa recebe uma solicitação de *serviço* ela muda o modo do display para *4v/2s*, mostrando seu **8.8.8.8** no lugar do seu cronômetro e ADC, e em seguida os dados do outro kit.

d.2) Será possível deixar de atender esse *serviço* (deixar de mostrar dados do outro e sair do modo *4v/2s*) acionando **A1** na sua placa. Isso faz voltar o modo de display para o formato *2v/4s* (item 'b' acima). Seu programa **deve enviar uma mensagem avisando que não está mais prestando o serviço**. Mas, se o outro kit enviar novamente a solicitação (ou seja, se pressionar **A2** no outro kit), sua placa tem que voltar a atender.

e) Claro que a placa do colega NÃO pode rejeitar uma solicitação de serviço se você acionar **A2** na sua placa. Portanto, ao acionar **A2**, **seu kit vai ter que enviar dados** para a placa do colega;

e.1) Se o outro kit estiver atendendo serviço e seu kit receber a mensagem de que o outro kit deixou de mostrar seus dados (item d.2), mostre no seu display a mensagem **n5Er** e pulse o *buzzer* por 5 segundos, indicando que será preciso reativar o serviço pressionando **A2** novamente.

f) quando o **A2** for acionado em AMBAS as placas, a **comunicação deve ocorrer de forma cruzada** – ou seja, sua placa mostra os dados da placa do colega e a placa dele mostra seus dados, ambas no formato *4v/2s*, como descrito no item 'd'.

g) ao acionarmos o botão **A3 (PA3)** seu kit **desiste de solicitar o serviço do kit do colega** (ou vice-versa, se o colega acionar **A3** no kit dele). Ao acionar **A3**, um kit libera o outro para voltar a mostrar só os dados locais (vice-versa). Portanto, ao acionar **A3** você **deve enviar uma mensagem para o outro kit informando que não requer mais o serviço** (vice-versa, você deve interpretar quando recebeu tal mensagem).

g.1) Obs1: se o colega acionar **A3** na outra placa, você deixa de mostrar o valor dele e voltará para o modo de amostragem local: se você ainda não tiver acionado **A1**, sua placa volta a mostrar **8.8.8.8**; mas se já estava mostrando seus dados, então volte ao modo de amostragem *2v/4s* descrito no item 'b'.

g.2) Obs2: o botão **A3 dispensa sua requisição de serviço ao outro kit**; mas, não libera seu kit de atender o serviço da outra placa se estiver ativo (diferente de quando desistimos da requisição de serviços acionando **A1**). Ou seja, se a outra acionar **A3**, a sua placa ficará liberada de mostrar os dados dele; mas se você acionar **A3**, e a outra placa continuar requisitando o serviço, seu kit deve mostrar os dados dela (como no item 'd').

h) Devemos evitar conflitos no uso de recursos; ou seja, se necessário use **semáforo, mutex, queue**, e outros recursos evitando assim perder mensagem, ou para evitar *race conditions* em *buffers, periféricos*, etc.

i) a turma (toda a classe) deve *analisar requisitos* e decidir COMO será a comunicação entre as placas:

i.1) que tipo de **dados** suas placas vão trocar?

i.2) qual '**protocolo**' será seguido durante a comunicação?

Algumas questões que, se vocês responderem, ajuda a analisar requisitos e decidir:

(1) Só começo enviar dados depois que a placa do outro solicitar? – *protocolo*!

(2) Ou envio dado '*direto*' sem me preocupar se os dados estão sendo recebidos? - *protocolo*

(3) Envio o dado cru lido no ADC? – *formato dos dados* – Ou envio o valor de cada dígito decimal? - *formato*

(4) Como enviar o valor do cronômetro? – *formato*

(5) Envio os dados em binário, floating, ou em algum código (e.g. ASCII)?

(6) Tenho que enviar mensagens [PING ("oper?", "oper!"), SERV("noREQ", "noSRV")] – como enviar?

(7) Como vou dividir minhas tarefas dentro do FreeRTOS?

Vocês discutem e decidem...

Temos tempo para elaborar o projeto e as aulas de LAB serão para dar suporte à criação do projeto. Serão apresentadas as rotinas para mostrar dados nos displays, para conversão ADC, e comunicação via UART.

Lembre-se: uma hora de planejamento elimina 10 horas de retrabalho!

**** você vai mostrar funcionando em sala de aula, e ** vai enviar o a pasta SRC do projeto zipada por e-mail.**

A nota depende de (a) programa funcionar, (b) da clareza do código, (c) da participação e presença do aluno no curso.